

Phase- and Amplitude-Preserving Normalization for Robust SAR Automatic Target Recognition

Gourav

PhaseSAR-Net Project · APS College of Engineering, Bengaluru · github.com/arisu2006/phase-aware-sar

Abstract

Synthetic Aperture Radar (SAR) imagery is complex-valued, combining an amplitude component with an extreme dynamic range and a phase component that is inherently circular. Naive preprocessing pipelines borrowed from optical computer vision typically discard phase entirely or normalize it as if it were a bounded scalar, both of which destroy information relevant to automatic target recognition (ATR) robustness. This report documents two preprocessing components built for PhaseSAR-Net: (1) a normalization scheme that log-compresses amplitude while preserving phase circularity via wrapped-angle and sine/cosine encoding, and (2) a dual tensor representation module offering both amplitude-phase (polar) and real-imaginary (Cartesian) views of a chip, selectable per experiment. Both components are validated numerically and visually, and are framed within the project's broader reliability/robustness question: does preserving phase, rather than discarding it, improve resilience of SAR classification under compression and corruption.

1. Introduction

Most publicly available SAR ATR pipelines (e.g. magnitude-only MSTAR baselines) discard the phase channel and operate purely on detected amplitude. This is convenient because amplitude behaves like a conventional single-channel image, but it discards the scattering-center phase information that governs how returns from different parts of a target interfere constructively or destructively. PhaseSAR-Net's central thesis is that retaining this phase information, if normalized correctly, should improve robustness to noise and compression relative to amplitude-only baselines. Correct normalization is a prerequisite for that comparison to be meaningful: if phase is corrupted by the preprocessing step itself, any downstream robustness result is confounded.

2. Normalization Methodology

2.1 Amplitude: Log-Compression

SAR amplitude values span several orders of magnitude between strong metallic returns and weak or absent returns from background clutter. Linear min-max scaling under this distribution pushes the vast majority of pixels toward zero, starving the network of gradient signal outside a handful of bright pixels. We instead apply $\log(1 + \text{amplitude})$, which compresses the dynamic range while remaining numerically stable at zero, then min-max scale the log-amplitude to $[0, 1]$ using either a per-chip or dataset-level maximum.

2.2 Phase: Circularity Preservation

Phase is an angular quantity: $-\pi$ and $+\pi$ denote the same physical angle, so treating phase as an ordinary bounded scalar and min-max scaling it directly introduces a false discontinuity at the wrap boundary. Our pipeline instead (a) keeps phase wrapped to $[-\pi, \pi]$ via `numpy.angle`, which is idempotent under repeated wrapping, and (b) exposes phase to the network as a $(\sin \theta, \cos \theta)$ pair rather than a raw scalar wherever a bounded, differentiable representation is needed. This encoding is circularity-preserving by construction and is fully invertible via `atan2`.

3. Dual Representation Design

A normalized complex chip $z = a + bi$ admits two equivalent tensor encodings. The **amplitude-phase (polar)** view stacks log-normalized amplitude with the sin/cos phase encoding into a 3-channel tensor and matches how the target framing (scattering strength and interference phase) is typically discussed. The **real-imaginary (Cartesian)** view stacks the real and imaginary parts directly into a 2-channel tensor and matches the arithmetic definition of complex convolution, $(a+bi)(c+di) = (ac-bd) + (ad+bc)i$, used by the project's ComplexConv2d layers. Both views are produced from the same normalization step and are selectable via a configuration flag, which will allow an ablation between representations in later training phases without changing the normalization logic.

4. Validation

Three checks were used to validate correctness before committing. First, `wrap_phase` was confirmed idempotent: re-wrapping an already-wrapped phase array leaves it unchanged. Second, the sin/cos encoding was confirmed to reconstruct the wrapped phase exactly via `atan2` (max reconstruction error $< 1e-4$ rad). Third, amplitude recovered from the real-imaginary representation, $\sqrt{\text{real}^2 + \text{imag}^2}$, was confirmed to match the amplitude channel of the polar representation to within float32 precision (max absolute difference $\approx 1.2 \times 10^{-7}$), confirming the two representations encode identical information.

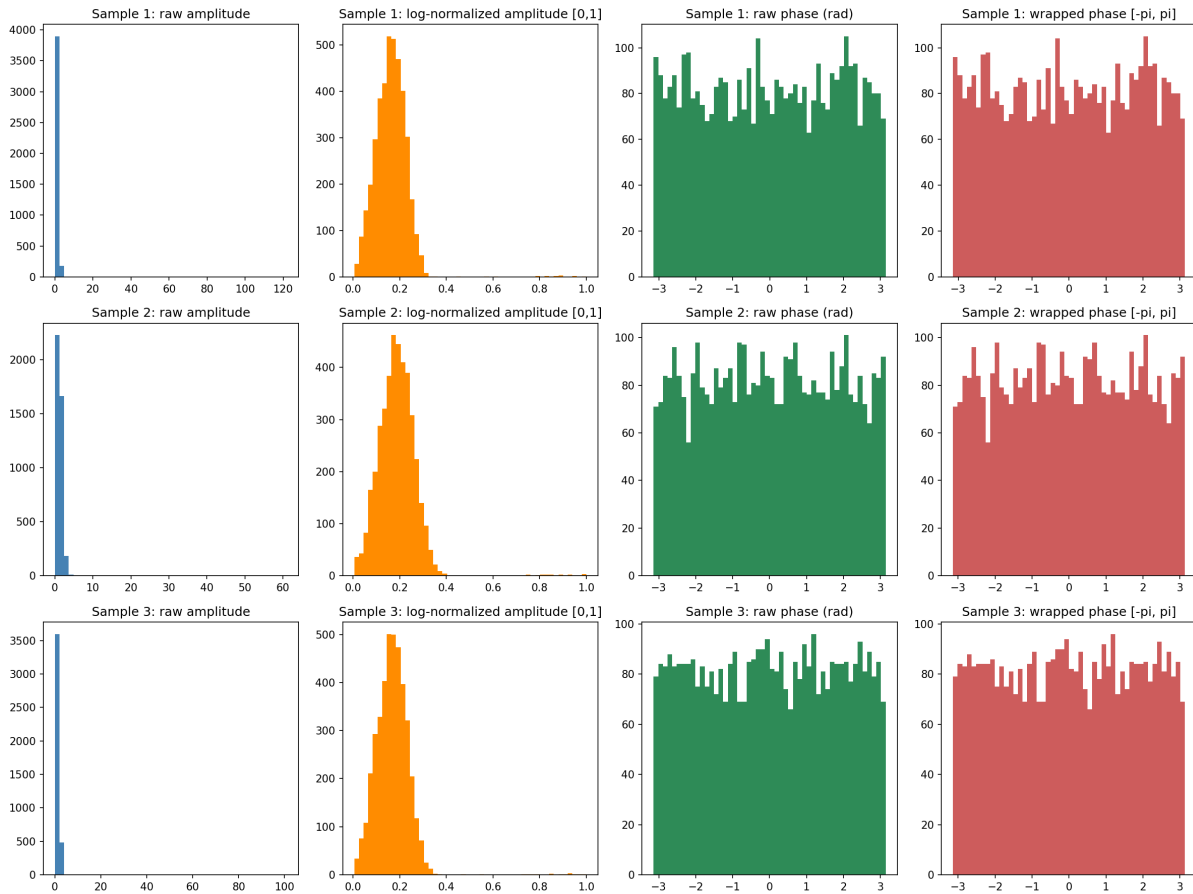


Figure 1. Raw vs. log-normalized amplitude and raw vs. wrapped phase, three synthetic sample chips. Log-compression redistributes amplitude mass away from near-zero; phase wrapping keeps the distribution bounded to $[-\pi, \pi]$.

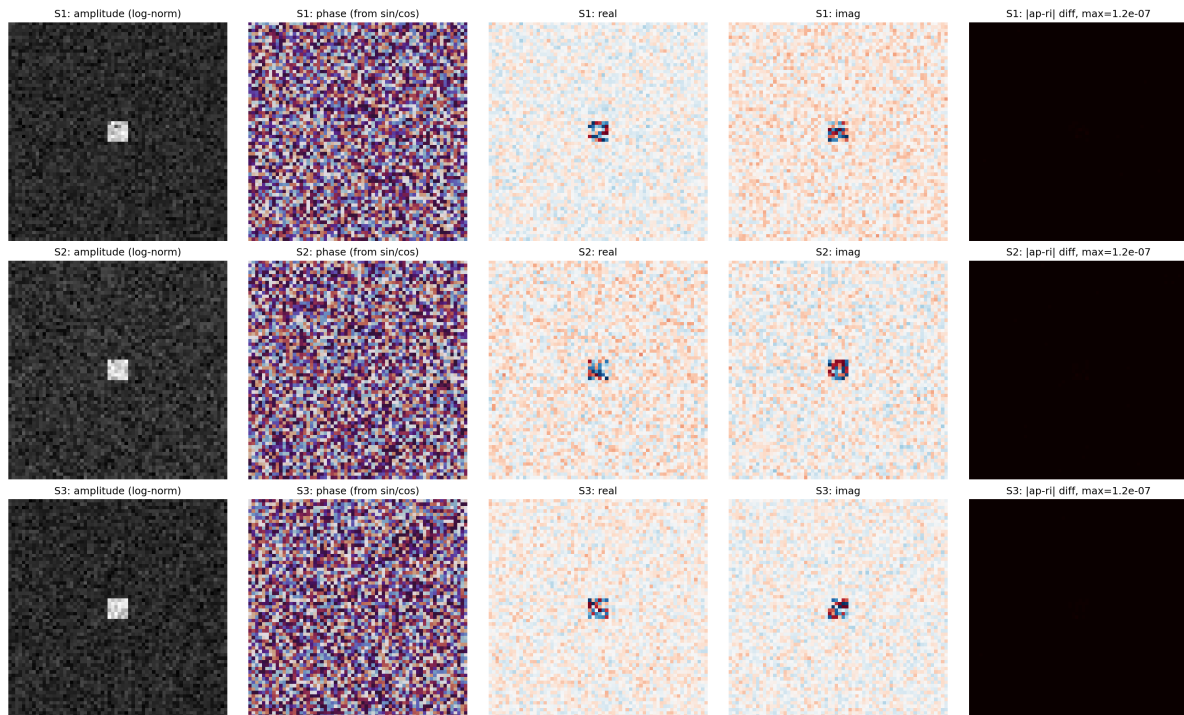


Figure 2. Amplitude, phase, real, and imaginary channels for three sample chips, with a fixed-scale consistency-check panel (rightmost column, $v^{\max}=1 \times 10^{-5}$) confirming the polar and Cartesian representations agree to floating-point precision.

5. Status and Next Steps

Date	Deliverable	Status
31 Aug 2026	normalize.py implemented	Complete
01 Sep 2026	normalize.py validated + histogram comparison	Complete
02 Sep 2026	representation.py + side-by-side comparison	Complete

The immediate open item is that both modules were validated on synthetic complex-valued chips rather than real MSTAR data, pending dataset access confirmation. Once access is confirmed, the placeholder loader in each validation script should be replaced with the real MSTAR chip loader; the normalization and representation logic itself requires no changes to accept real data. Phase 3 concludes on 06 Sep 2026 with full docstring documentation, a regeneration guide in data/README.md, and the tagged release v0.3-preprocessing-complete.